Assignment 2 Reflection

Name: Prem Kumar Mani
Option: B – Tasks
Date: 10/11/2026

Tool Design Decisions

Tools Implemented

1. **list_tasks:** Retrieves the user's Google Tasks and their current status.
2. **create_task:** Creates a new task with the specified details.
3. **delete_task:** Deletes an existing task using its task ID.
4. **complete_task:** Marks an existing task as completed.
5. **update_task:** Updates task details, such as the title or description.

In addition to the Google Tasks tools, I integrated GitHub through an MCP server and experimented with dynamic tool discovery using `search_github_tools` and `call_github_tool`.

Why These Tools?

I initially started implementing Google Calendar integration. However, handling calendar invitations with multiple attendees required additional implementation and testing time. Due to time constraints, I decided to focus on Google Tasks.

I selected these five tools because they cover the basic task management lifecycle: creating, retrieving, updating, completing, and deleting tasks. This allowed me to build a functional assistant while concentrating on Google ADK, tool integration, and agent behavior.

I also wanted to explore whether a single agent could support both Google Tasks and GitHub operations.

Description Strategy

I used descriptive tool names and clear function descriptions so the LLM could identify the appropriate tool based on the user's intent.

For example, keywords such as *list*, *create*, *update*, *complete*, and *delete* map directly to the corresponding operations.

I also defined instructions to distinguish between operations that retrieve information and those that modify data. For updates and deletions, the agent is instructed to identify the correct task ID before proceeding.

For GitHub integration, I used tool descriptions and MCP input schemas to help the agent discover the appropriate operation and supply the required arguments.

Challenges Encountered

Challenge 1: Google ADK Package Structure and Imports

Problem: During the initial setup, ADK could not resolve certain modules, including configuration and tool packages. I encountered import errors because the application was structured as a Python package, but some imports referenced modules without their package prefixes.

Solution: I reorganized the imports using fully qualified paths, such as `workspace_assistant.config.settings` and `workspace_assistant.tools.tasks_tools`. This helped ensure that the modules were resolved correctly when running the application through ADK.

Challenge 2: Unsupported `defer_loading` Parameter

Problem: While implementing the optional tool-search optimization, I discovered that `defer_loading=True` was not supported by the installed Google ADK version. Passing the parameter to `McpToolset` resulted in a constructor error.

Solution: Instead of relying on the unsupported parameter, I explored a custom implementation using `tool_filter` to expose frequently used GitHub tools and separate `search_github_tools` and `call_github_tool` functions for discovering and executing other operations.

This allowed me to demonstrate an on-demand tool discovery approach without modifying the ADK framework itself.

Challenge 3: Supporting Filtered and Unfiltered MCP Tools

Problem: When I introduced dynamic tool search, the agent sometimes entered a cycle of repeatedly searching for tools or executing operations without returning a final response.

Solution: I refined the agent instructions to prioritize directly available tools. If a suitable tool was not already available, the agent could search for the required operation and execute it.

I also instructed the agent to avoid repeated searches for the same request and to return a final response after obtaining a satisfactory result.

This helped me understand the importance of controlling tool selection and defining clear completion conditions for agent workflows.

Challenge 4: Combining Google Tasks and GitHub in One Agent

Problem: Google Tasks and GitHub represent two different functional domains. Ideally, they could be implemented as separate specialized agents, but I wanted to explore whether a single agent could handle both.

The challenge was ensuring that the agent selected the correct toolset and did not use Google Tasks tools for GitHub requests or vice versa.

Solution: I created separate sections in the agent instructions for Google Tasks and GitHub operations.

I also experimented with different agent configurations to compare standard MCP tool loading with the tool-search approach.

This helped me understand both the flexibility and the limitations of combining multiple domains within a single agent.

Error Handling Approach

I approached error handling at both the tool level and the agent instruction level.

At the tool level, I considered failures such as missing credentials, invalid task IDs, Google API permission errors, MCP connection failures, invalid tool arguments, and unsuccessful tool execution.

For Google Tasks, I used task IDs to identify records before performing updates or deletions. I also instructed the agent to request confirmation before deleting a task.

For GitHub MCP operations, I added exception handling and structured execution responses so that the agent could distinguish successful operations from failures.

At the agent level, I instructed the LLM to communicate errors clearly, avoid inventing task IDs or repository information, and avoid repeatedly executing the same failed operation.

Ideas for Improvement

1. **Separate specialized agents:** Create independent Google Tasks and GitHub agents, coordinated by a router or parent agent. This would improve separation of responsibilities and simplify tool selection.
2. **Improve dynamic tool discovery:** Extend the MCP tool-search implementation with better tool ranking, schema validation, and programmatically enforced execution limits. I would also measure prompt token usage to evaluate whether the approach provides meaningful context savings.

Key Learnings

Tool descriptions, function schemas, application structure, and clear agent instructions all influence whether the agent selects and executes the correct operation. Integrating Google Tasks gave me hands-on experience with external APIs, while GitHub MCP integration helped me understand how agents can discover and invoke tools through a standardized interface.

Token Usage and Context Optimization

One of my key learnings was understanding how tool definitions and MCP integration impact LLM token consumption. When all GitHub MCP tools are loaded, their descriptions and input schemas increase prompt token usage, even when only a few tools are needed for a specific request.

I experimented with `tool_filter` and preloaded frequently used tools to reduce the number of tool schemas exposed to the model.

Token Usage Comparison

Both configurations used `gemini-3-flash-preview`.

Configuration	Prompt Tokens
Without tool filtering (all MCP tools loaded)	7,460
With tool filtering and selected preloaded tools	1,180
Reduction	**6,280 (84.2%)**

The filtered configuration showed an **84.2% reduction in prompt tokens** for the measured model requests. This demonstrates how limiting the tools exposed to an LLM can significantly reduce prompt context overhead.

I also learned how to inspect `promptTokenCount`, `candidatesTokenCount`, and `totalTokenCount` using Google ADK execution traces.

An important takeaway was that token optimization requires balancing tool availability with context efficiency. Although filtering reduced prompt tokens substantially in this comparison, a complete performance evaluation should measure token consumption across all model calls, including additional tool discovery and execution steps.

This exercise helped me understand why **tool filtering, dynamic tool discovery, and token usage monitoring** are important when designing scalable, cost-efficient agentic AI applications.

Finally, I learned that reliability, security, and observability are essential when developing agentic applications. Repeated tool calls, authentication failures, unexpected response structures demonstrated the importance of validation, structured error handling, and secure development practices. Going forward, I would focus on separating agents by responsibility, enforcing execution boundaries programmatically to make the overall system more predictable and production-ready.